

CPSC 332 Group Project

Pizza Restaurant Chain Database

Professor Anli Ji

Participants:

- Cameron Abo
- Charles Navarro

Prepared for CPSC 332

Date: May 8, 2026

Table of Contents

Mini-World	3
Requirement Analysis	4-6
ER/EER Model	7
Relational Model	8
Normalization	9-10
Data Dictionary	11-18
Summary	19

Mini-World

The environment being simulated for the Database being created is based on a State-Wide Pizza chain with multiple store fronts throughout the state of California. The database needs to be able to efficiently organize the data of all locations, employees, and order transactions, so the business administration can make decisions based on reliable and clear data that has been collected. The simulated business will also have an online ordering system for deliveries and takeout orders and must be able to keep track of customer accounts along with related data including purchase history, primary store locations, and saved payment methods.

This business is a non-franchise, centralized business and therefore is in charge of restocking and managing all store locations. Therefore, the database needs to provide accurate data to store inventory stocks and what items are being sold in order to accurately restock ingredients for products. It is important to understand trends of specific locations or special promotions to ensure that all stores are fully operational, but also to minimize the amount of waste and leftover products. It is very important to have a clear separation between store locations and the relation to other entities since that will be the main focus on data readability.

Requirement Analysis

Data Requirements

1. EMPLOYEE
 - a. SSN, Name, HireData, Role, Wage, Phone
2. STORE
 - a. storeID, phone, city, address
3. INVENTORY_ITEM
 - a. InventoryID, ItemName, ItemType, Unit, QuantityOnHand, ReorderLevel
4. PRODUCT
 - a. ProductID, ProductName, Category, Price
5. ORDER
 - a. OrderID, OrderDateTime, Total, Status
6. DINING_ORDER
 - a. PartySize, TableNumber
7. TAKEOUT_ORDER
 - a. PickupTime
8. DELIVERY_ORDER
 - a. DeliveryAddress, EstimatedArrivalTime
9. ORDER_ITEM
 - a. OrderID, ProductID, Quantity, ItemPrice, SpecialInstructions
10. CUSTOMER
 - a. CustomerID, Name, Phone, Email
11. PAYMENT
 - a. PaymentID, Amount, PaymentDateTime
12. PAYMENT_METHOD
 - a. PaymentMethodID, PaymentType, LastFourDigits

Constraints

1. Entity Constraints
 - a. Each table will have a primary key to uniquely identify each record.
2. Domain Constraints
 - a. Pizza price must be > 0
 - b. Quantity must be > 0
 - c. Employee wage > 0
 - d. Order total ≥ 0

- e. Party size > 0

3. Unique Constraints

- a. Restaurant location address must be unique
- b. Restaurant phone number must be unique
- c. Pizza name and size combination must be unique
- d. Topping name must be unique
- e. Customer email must be unique

4. Semantic Constraints

- a. The customer can place multiple orders, but each order belongs to one customer.
- b. Each order must contain at least one pizza.
- c. An order is placed at one location, but the location can process multiple orders.
- d. The pizza can have multiple toppings and appear in multiple orders.
- e. Toppings can be used on multiple pizzas.
- f. Each location can employ multiple employees, but each employee works at one location.
- g. Each order must have one associated payment record.
- h. A pizza cannot contain the same topping more than once.

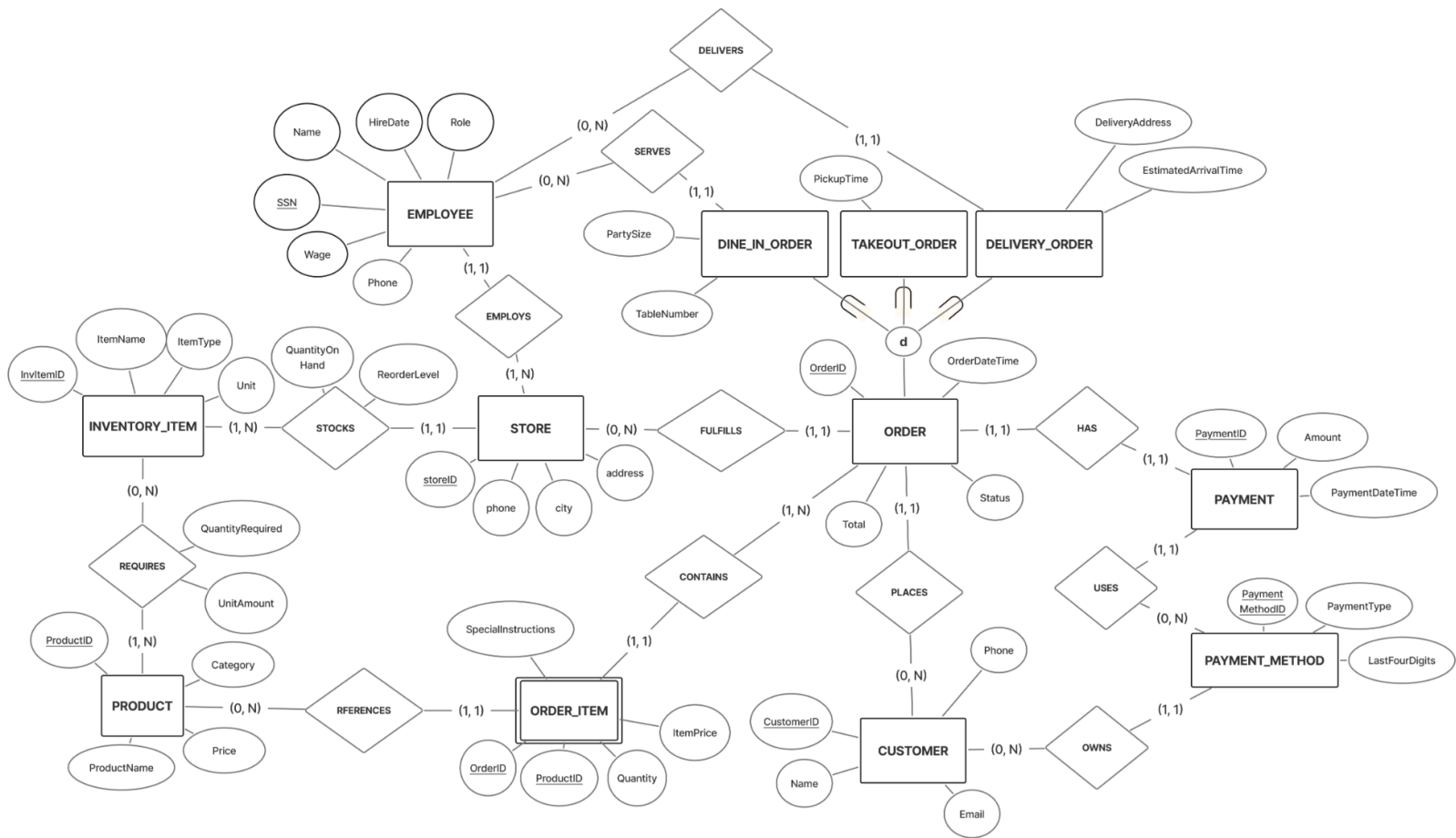
Functional Requirements

1. View all the restaurant locations across the pizza chain.
2. Display a full menu, including pizza names and toppings.
3. Create and record new customer orders.
4. Record payment information for completed orders.
5. Manage inventory items for each store.
6. View customer information and order history.

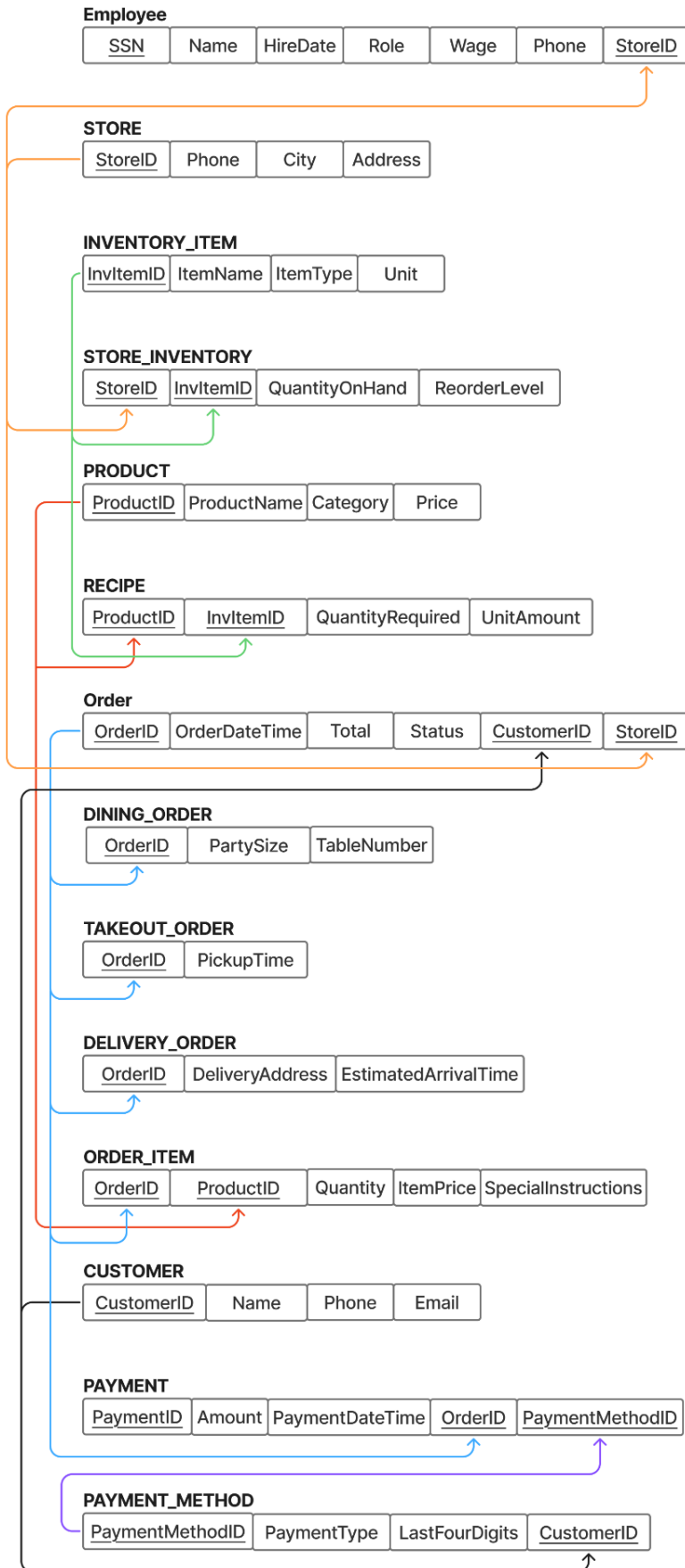
Non-Trivial Functional Requirements

1. Show all the orders placed by the customer.
2. Display pizzas and toppings included in each order.
3. Calculate the total cost of each order based on items selected.
4. Calculate total revenue by each restaurant location.
5. Display total amount of employees working at each specific restaurant location.
6. Track inventory levels and identify low stock items
7. Generate reports for most popular pizzas ordered and toppings used.

EER Diagram



Relational Diagram

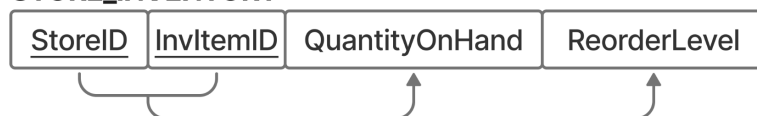


Normalization

The relational schema currently qualifies as 3nf and normalizing it any further would create readability issues for the company administrators. This schema allows for just the use of naming entities in queries instead of separate relational tables in order to provide accurate data from simple queries. The only entities that really need to be examined are STORE_INVENTORY, RECIPE, and ORDER_ITEM since those entities are the only ones with composite keys.

STORE_INVENTORY has all atomic attributes, and QuantityRequired and UnitAmount depend on the composite key of StoreID and InvItemID making it qualify as 1NF and 2NF. There are also no transitive dependencies which also qualifies STORE_INVENTORY as 3NF.

STORE_INVENTORY

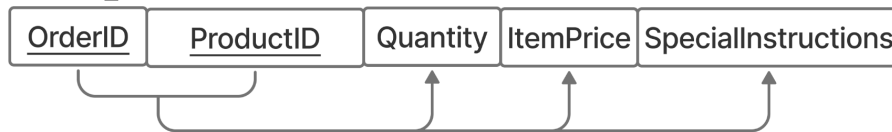
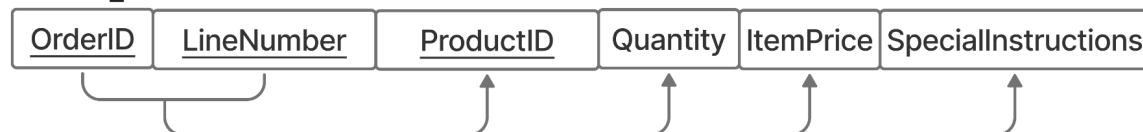


The same can also be said for the RECIPE entity. Each attribute is atomic, the non-key attributes depend on the composite key, and there are no transitive dependencies. Therefore, RECIPE also qualifies as 3NF.

RECIPE



ORDER_ITEM needs another candidate key to replace ProductID, so LineNumber will be added to be part of the new primary key of ORDER_ITEM. This change allows for an ORDER to contain duplicate items with different special instructions. Say, for example, a customer wants to order two pepperoni pizzas but one without sauce. The old schema would cause a conflict and would not allow for it since the primary keys would be duplicated and no longer unique. The LineNumber candidate key fixes this issue by assigning the primary key to each ORDER_ITEM based on the sequence it was added to the ORDER rather than the ProductID which is a fixed value within the ordering process.

ORDER_ITEM**ORDER_ITEM**

Data Dictionary

EMPLOYEE

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
SSN	CHAR(9)	PK, CK	NOT NULL, UNIQUE, must contain 9 digits	Uniquely identifies each employee.
Name	VARCHAR(100)		NOT NULL	Stores the employee's full name.
HireDate	DATE		NOT NULL, CHECK HireDate	Stores the date the employee was hired.
Role	CARCHAR(50)		NOT NULL	Stores the employee's position, such as cashier, cook, driver, or manager.
Wage	DECIMAL(6,2)		NOT NULL, CHECK WAGE > 0	Stores the employee's hourly pay rate.
Phone	VARCHAR(15)	CK	UNIQUE	Stores the employee's phone number.
StoreID	INT	FK	NOT NULL	Identifies the store where the employee works.

STORE

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
StoreID	INT	PK, CK	NOT NULL, UNIQUE	Uniquely identifies each store location.
Phone	VARCHAR(15)	CK	NOT NULL, UNIQUE	Stores the phone number for the store.
City	VARCHAR(50)		NOT NULL	Stores the city where the store is located.
Address	VARCHAR(150)	CK	NOT NULL, UNIQUE	Stores the physical address of the store.

INVENTORY_ITEM

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
InvItemID	Int	PK, CK	NOT NULL, UNIQUE	Uniquely identifies each inventory item.
ItemName	VARCHAR(100)	CK	NOT NULL	Stores the name of an inventory item.
ItemType	VARCHAR(50)		NOT NULL	Describes the type of inventory item
Unit	VARCHAR(25)		NOT NULL	Describes the unit used to measure the item

STORE_INVENTORY

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
StoreID	Int	PK, FK	NOT NULL	Identifies the store holding the inventory item.
InvItemID	Int	PK, FK	NOT NULL	Identifies the inventory item being tracked.
QuantityOnHand	Decimal(8,2)		NOT NULL, CHECK QuantityOnHand >= 0	Stores the amount of the item currently available at the store.
ReorderLevel	Decimal(8,2)		NOT NULL, CHECK ReorderLevel >= 0	Stores the minimum amount before the item should be reordered.

PRODUCT

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
ProductID	INT	PK, CK	NOT NULL, UNIQUE	Uniquely identifies each menu product.
ProductName	VARCHAR(100)	CK	NOT NULL	Stores the name of the menu product.
Category	VARCHAR(50)	CK	NOT NULL	Stores the product category, such as pizza, drink, side, or dessert.
Price	DECIMAL(8,2)		NOT NULL, CHECK Price > 0	Stores the selling price of the product.

RECIPE

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
ProductID	INT	PK, FK	NOT NULL	Identifies the product being made.
InvItemID	INT	PK, FK	NOT NULL	Identifies the inventory item required for the product.
QuantityRequired	DECIMAL(8,2)		NOT NULL, CHECK QuantityRequired > 0	Stores how much of the inventory item is needed to make the product.
UnitAmount	VARCHAR(25)		NOT NULL	Stores the measurement unit for the recipe amount.

CUSTOMER

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
CustomerID	INT	PK, CK	NOT NULL, UNIQUE	Uniquely identifies each customer.
Name	VARCHAR(100)		NOT NULL	Stores the customer's full name.
Phone	VARCHAR(15)	CK	UNIQUE	Stores the customer's phone number.
Email	VARCHAR(100)	CK	NOT NULL, UNIQUE	Stores the customer's email address.

ORDER

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
OrderID	INT	PK, CK	NOT NULL, UNIQUE	Uniquely identifies each order.
OrderDateTime	DATETIME		NOT NULL	Stores the date and time the order was placed.
Total	DECIMAL(8,2)		NOT NULL, CHECK Total >= 0	Stores the final total cost of the order.
Status	VARCHAR(30)		NOT NULL, CHECK Status IN ('Pending', 'Preparing', 'Completed', 'Canceled', 'Delivered')	Stores the current status of the order.
CustomerID	INT	FK	NOT NULL	Identifies the customer who placed the order.
StoreID	INT	FK	NOT NULL	Identifies the store that processed the order.

DINING_ORDER

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
OrderID	INT	PK, FK	NOT NULL	Identifies the order as a dine-in order.
PartySize	INT		NOT NULL, CHECK PartySize > 0	Stores the number of people dining.

TableNumber	INT		NOT NULL, CHECK TableNumber > 0	Stores the table number assigned to the order.
-------------	-----	--	---------------------------------------	--

TAKEOUT_ORDER

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
OrderID	INT	PK, FK	NOT NULL	Identifies the order as a takeout order.
PickupTime	DATETIME		NOT NULL, CHECK PickupTime >= OrderDateTime	Stores the expected pickup time for the customer.

DELIVERY_ORDER

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
OrderID	INT	PK, FK	NOT NULL	Identifies the order as a delivery order.
DeliveryAddress	VARCHAR(150)		NOT NULL	Stores the delivery destination address.
EstimatedArrival Time	DATETIME		NOT NULL, CHECK EstimatedArrival Time >= OrderDateTime	Stores the expected arrival time for the delivery.

ORDER_ITEM

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
OrderID	INT	PK, FK	NOT NULL	Identifies which order the item belongs to.
LineNumber	INT	PK	NOT NULL, CHECK LineNumber > 0	Identifies the item line number within an order.
ProductID	INT	FK	NOT NULL	Identifies the product ordered.
Quantity	INT		NOT NULL, CHECK Quantity > 0	Stores how many of the product were ordered.
ItemPrice	DECIMAL(8,2)		NOT NULL, CHECK ItemPrice >= 0	Stores the product price at the time of purchase.
SpecialInstructions	VARCHAR(255)		UNIQUE	Stores optional customer instructions, such as no sauce or extra cheese.

PAYMENT

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
PaymentID	INT	PK, CK	NOT NULL, UNIQUE	Uniquely identifies each payment.
Amount	DECIMAL(8,2)		NOT NULL, CHECK Amount >= 0	Stores the payment amount.
PaymentDateTime	DATETIME		NOT NULL	Stores the date

				and time the payment was made.
OrderID	INT	FK, CK	NOT NULL, UNIQUE,	Identifies the order paid for.
PaymentMethodID	INT	FK	NOT NULL	Identifies the payment method used.

PAYMENT_METHOD

Attribute	Domain / Data Type	Keys	Constraints	Semantic Interpretation
PaymentMethodID	INT	PK, CK	NOT NULL, UNIQUE	Uniquely identifies each saved payment method.
PaymentType	VARCHAR(30)		NOT NULL, CHECK PaymentType IN ('Cash', 'Credit Card', 'Debit Card', 'Gift Card', 'Online Payment')	Stores the type of payment method.
LastFourDigits	CHAR(4)		CHECK four digits when card-based	Stores the last four digits of the card, when applicable.
CustomerID	INT	FK	NOT NULL	Identifies the customer who owns the payment method.

Summary

For our group project we created a database for a pizza restaurant chain with multiple store locations. The main goal of the database is to organize important business information such as stores, employees, customers, orders, payments, products, and inventory. Since the company is centralized and not a franchise, the database also helps track inventory levels at each store so the business can know when items need to be restocked.

The project includes a requirement analysis, EER diagram, relational diagram, normalization section, and data dictionary. The database supports different types of orders, including dine in, takeout, and delivery. It also keeps track of order items, payment methods, customer information, and product recipes.

Overall, this database is designed to help the pizza chain manage daily operations, view customer order history, calculate revenue, track inventory, and make better business decisions using organized data.